MyCodeSchool

Exerciții — șiruri de caractere (3)

Python · proiectul `py-siruri` · Luca

Ce ai voie: tot din fișele 1 și 2 · felii · `for litera in text`, `for i in range(1, len(text))` cu `text[i]` și `text[i - 1]` · `.split()`, `.split(",")`, `bucati = text.split(".")` și apoi `bucati[0]` · contor, acumulator, maxim/minim · `in`, `not in` · `int()`, `str()`, f-string · `.lower()`, `.count()`.
Două noțiuni noi (explicate mai jos): `.find()` și `.isdigit()`.
Încă NU folosim: liste construite de tine, dicționare, `.join()`, `print` **în funcții**.

Cum lucrezi + cele două noțiuni noi

Scrii în `siruri3.py`. Fiecare funcție întoarce cu `return`; `main.py` afișează:

```
from siruri3 import utilizator_din_email

def main():
    print(utilizator_din_email("luca@mail.com"))
    print(utilizator_din_email("a@b.ro"))

if __name__ == "__main__":
    main()
```

Ieșire așteptată: `luca` și, pe rândul următor, `a`. La fiecare funcție nouă: o adaugi în `import` și pui câte un `print` pentru fiecare rând din tabelul ei.

`text.find(x)` întoarce **poziția** primei apariții a lui `x`, iar dacă nu există, întoarce `-1` (nu dă eroare):

```
"luca@mail.com".find("@")   → 4        "@mail.com".find("@")   → 0
"luca".find("@")            → -1
```

`x.isdigit()` e `True` doar dacă textul are cel puțin un caracter și **toate** sunt cifre:

```
"7".isdigit()  → True     "123".isdigit()  → True     "12a".isdigit()  → False
"a".isdigit()  → False    "".isdigit()  → False
```

La clasă

Numele din adresa de email — 1

```
def utilizator_din_email(email: str) -> str:
```

Întoarce tot ce e **înainte** de `@`. Dacă adresa nu are `@`, întoarce `""`.

apel	întoarce
`utilizator_din_email("luca@mail.com")`	`"luca"`
`utilizator_din_email("luca.todor@yahoo.com")`	`"luca.todor"`
`utilizator_din_email("a@b.ro")`	`"a"`
`utilizator_din_email("@mail.com")`	`""`
`utilizator_din_email("luca")`	`""`
`utilizator_din_email("")`	`""`

Poziția o iei cu `.find("@")`, apoi o tai cu o felie care se oprește acolo. Rulează `"luca"[:-1]` în consolă și vezi ce ar ieși dacă ai folosi `-1` direct ca limită: de aceea ai nevoie de un `if`.

Cifrele dintr-o adresă — 2

```
def suma_cifrelor_din_text(text: str) -> int:
```

Adună doar cifrele din text; restul caracterelor le ignori.

apel	întoarce
`suma_cifrelor_din_text("Bloc 5, ap. 12")`	`8`
`suma_cifrelor_din_text("Ana are 3 mere si 4 pere")`	`7`
`suma_cifrelor_din_text("9")`	`9`
`suma_cifrelor_din_text("Strada Libertatii")`	`0`
`suma_cifrelor_din_text("")`	`0`

Ca la `suma_cifrelor` din fișa 1, dar nu toate literele sunt cifre. Întrebarea din `if` o pune `litera.isdigit()`.

Data în format ISO 3

```
def data_iso(data: str) -> str:
```

Primești `zi.luna.an`, cu zi și lună scrise cu una sau două cifre. Întoarce `an-luna-zi`, cu ziua și luna **mereu** pe două cifre. Pentru `""` întoarce `""`.

apel	întoarce
`data_iso("21.10.2026")`	`"2026-10-21"`
`data_iso("5.3.2027")`	`"2027-03-05"`
`data_iso("1.12.2026")`	`"2026-12-01"`
`data_iso("31.1.2027")`	`"2027-01-31"`
`data_iso("")`	`""`

`bucati = data.split(".")`, apoi `bucati[0]`, `bucati[1]`, `bucati[2]`. Un număr cu o singură cifră are lungimea 1: ce îi adaugi în față?

Mai grele

Cea mai caldă zi 4

```
def ziua_cea_mai_calda(temperaturi: str) -> int:
```

Temperaturile zilnice vin separate prin virgulă, pot fi negative. Întoarce **numărul zilei** (prima zi e 1) cu temperatura cea mai mare. La egalitate, ziua **mai devreme**. Fără zile: `0`.

apel	întoarce
`ziua_cea_mai_calda("21,25,19")`	`2`
`ziua_cea_mai_calda("-5,-3,-8")`	`2`
`ziua_cea_mai_calda("9,2,5")`	`1`
`ziua_cea_mai_calda("3,7,7")`	`2`
`ziua_cea_mai_calda("-4")`	`1`
`ziua_cea_mai_calda("0,-1,-2")`	`1`
`ziua_cea_mai_calda("")`	`0`

*Ai nevoie de **doi** acumulatori: maximul și ziua lui, plus un contor al zilei curente. Cu ce valoare pornește maximul, ca rândul `"-5,-3,-8"` să iasă bine?*

Număr de telefon valid 5

```
def telefon_valid(numar: str) -> bool:
```

Valid: exact 10 caractere, începe cu `07` și are **numai cifre** (fără spații).

apel	întoarce
`telefon_valid("0722123456")`	`True`
`telefon_valid("0622123456")`	`False`
`telefon_valid("072212345")`	`False`
`telefon_valid("07221234567")`	`False`
`telefon_valid("0722 12345")`	`False`
`telefon_valid("07221234ab")`	`False`
`telefon_valid("")`	`False`

*Primele două caractere sunt o **felie**, nu o literă: compară `numar[:2]` cu `"07"`. Pentru cifre, `.isdigit()` merge și pe tot textul deodată.*

Cel mai ieftin produs 6

```
def cel_mai_ieftin(produse: str) -> str:
```

Produsele sunt scrise `nume:pret`, separate prin virgulă. Întoarce **numele** celui mai ieftin; la același preț, cel de **mai devreme**. Fără produse: `""`.

apel	întoarce
`cel_mai_ieftin("paine:3,lapte:7,ou:1")`	`"ou"`
`cel_mai_ieftin("paine:2,lapte:7,ou:5")`	`"paine"`
`cel_mai_ieftin("lapte:4,ou:4")`	`"lapte"`
`cel_mai_ieftin("mere:5")`	`"mere"`
`cel_mai_ieftin("")`	`""`

Două `split` unul în altul: întâi `split(",")` pe tot textul, apoi `split(":")` pe fiecare produs. Prețul e text până îl treci prin `int()`. Minimul nu pornește de la `0`; ia prima valoare reală.

Caractere diferite într-o parolă 7

```
def caractere_diferite(text: str) -> int:
```

Câte caractere **distincte** are textul. Literele mari și mici sunt aceeași literă (`A` = `a`).

apel	întoarce
`caractere_diferite("banana")`	3
`caractere_diferite("Anna")`	2
`caractere_diferite("abcABC")`	3
`caractere_diferite("Parola12")`	7
`caractere_diferite("aaaa")`	1
`caractere_diferite("a")`	1
`caractere_diferite("")`	0

Ține un text `vazute = ""` . Adaugi litera în el doar dacă `not in vazute` . La final, ce îți spune `len(vazute)` ?

Provocări

Fără indicii.

Apariții care se suprapun 8

```
def numara_aparitii(text: str, bucata: str) -> int:
```

De câte ori apare `bucata` în `text` , **cu suprapuneri**: în `"aaaa"` , `"aa"` apare de 3 ori. Aici `.count()` nu ajunge (`"banana".count("ana")` dă 1, tu trebuie să întorci 2). Literele mari și mici sunt diferite; `bucata` nu e niciodată goală.

apel	întoarce
`numara_aparitii("banana", "ana")`	2
`numara_aparitii("aaaa", "aa")`	3
`numara_aparitii("abcabc", "bc")`	2
`numara_aparitii("abc", "x")`	0
`numara_aparitii("a", "a")`	1
`numara_aparitii("ab", "abc")`	0
`numara_aparitii("", "a")`	0

Comprimarea unui text 9

```
def comprimare(text: str) -> str:
```

Fiecare grup de litere identice, una după alta, devine litera + câte sunt în grup.

apel	întoarce
`comprimare("aaabcc")`	`"a3b1c2"`
`comprimare("aabaa")`	`"a2b1a2"`
`comprimare("abc")`	`"a1b1c1"`
`comprimare("zzzzzzzzzzz")`	`"z12"`
`comprimare("a")`	`"a1"`
`comprimare("")`	`""`

Suma numerelor din text 10

```
def suma_numerelor_din_text(text: str) -> int:
```

În text apar numere întregi pozitive, cu una sau mai multe cifre. Întoarce suma lor: `12` e un singur număr, nu `1` și `2` .

apel	întoarce
`suma_numerelor_din_text("Am 12 mere si 7 pere, 100 lei")`	119
`suma_numerelor_din_text("Camera 12, etaj 3")`	15
`suma_numerelor_din_text("a1b2c3")`	6
`suma_numerelor_din_text("7 mere")`	7
`suma_numerelor_din_text("x5")`	5
`suma_numerelor_din_text("2026")`	2026
`suma_numerelor_din_text("fara numere")`	0
`suma_numerelor_din_text("")`	0

Înainte să zici gata, la fiecare funcție:
1. Un `print` pentru **fiecare rând** din tabel; ieșirea e identică cu „întoarce”. Fără `None` .
2. Încearcă-o pe `""` , pe un text de **un singur caracter** și pe un caz în care răspunsul e **primul** element.
3. Valoarea de pornire (maxim, minim, ultimul grup) e una care poate apărea în date?